

C Problem Solving – Fixed Codes

Clean & Corrected Versions

All codes cleaned from PDF line-wrapping issues and verified.

Arrays/sum_array.c

```
#include <stdio.h>

int sum_array(const int arr[], int size) {
    int sum = 0;
    for (int i = 0; i < size; ++i)
        sum += arr[i];
    return sum;
}

int main(void) {
    int size;
    printf("Enter the size of the array: ");
    scanf("%d", &size);
    if (size <= 0) return 0;

    int arr[size];
    printf("Enter %d elements: ", size);
    for (int i = 0; i < size; ++i)
        scanf("%d", &arr[i]);

    printf("Sum = %d\n", sum_array(arr, size));
    return 0;
}
```

Arrays/three_dimensional_array.c

```
#include <stdio.h>

int main(void) {
    int x[3][3][3];

    for (int i = 0; i < 3; ++i)
        for (int j = 0; j < 3; ++j)
            for (int k = 0; k < 3; ++k)
                scanf("%d", &x[i][j][k]);

    for (int i = 0; i < 3; ++i)
```

```

        for (int j = 0; j < 3; ++j)
            for (int k = 0; k < 3; ++k)
                printf("%d%c", x[i][j][k],
                    (i == 2 && j == 2 && k == 2) ? '\n' : ' ');
    }

```

Files/generate_website.c

```

#include <stdio.h>

int main(void) {
    const char *html_start = "<html><head><title>";
    const char *html_middle = "</title><link rel=\"stylesheet\" href=\"style.cs";
    const char *html_end = "</body></html>";

    char title[100], body[500], color[30];
    printf("Enter the website title: ");
    scanf(" %99[^\n]", title);
    printf("Enter the website body: ");
    scanf(" %499[^\n]", body);
    printf("Enter the background color: ");
    scanf(" %29s", color);

    FILE *out = fopen("index.html", "w");
    FILE *css = fopen("style.css", "w");
    if (!out || !css) {
        if (out) fclose(out);
        if (css) fclose(css);
        return 1;
    }

    fprintf(out, "%s%s%s%s%s\n", html_start, title, html_middle, body, html_end);
    fprintf(css, "body { background-color: %s; }\n", color);

    fclose(out);
    fclose(css);
    return 0;
}

```

Files/grades_average.c

```

#include <stdio.h>

int main(void) {
    FILE *file = fopen("grades.txt", "r");
    if (file == NULL) {
        perror("grades.txt");
        return 1;
    }
}

```

```

int grade, sum = 0, count = 0;
while (fscanf(file, "%d", &grade) == 1) {
    sum += grade;
    ++count;
}
fclose(file);

if (count == 0) {
    printf("No grades found.\n");
    return 0;
}

printf("Average = %.2f\n", (double)sum / count);
return 0;
}

```

Functions/employee_tasks.c

```

#include <stdio.h>

int add_tasks(int tasks, int total) {
    return total + tasks;
}

int main(void) {
    int employees, total = 0, tasks;
    printf("Enter the number of employees: ");
    scanf("%d", &employees);

    for (int i = 0; i < employees; ++i) {
        printf("Enter tasks completed by employee %d: ", i + 1);
        scanf("%d", &tasks);
        total = add_tasks(tasks, total);
    }

    printf("Total tasks completed by all employees: %d\n", total);
    return 0;
}

```

Linked-List/doubly_linked_list.c

```

#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node *next;
    struct Node *prev;
};

```

```

static struct Node *create(int data) {
    struct Node *node = malloc(sizeof(struct Node));
    if (node == NULL) return NULL;
    node->data = data;
    node->next = NULL;
    node->prev = NULL;
    return node;
}

static struct Node *insert(struct Node *head, int data) {
    if (head == NULL) return create(data);
    head->next = insert(head->next, data);
    if (head->next != NULL)
        head->next->prev = head;
    return head;
}

static void display(const struct Node *head) {
    printf("Double Linked List: ");
    while (head != NULL) {
        printf("%d", head->data);
        if (head->next != NULL) printf("<->");
        head = head->next;
    }
    printf("<->NULL\n");
}

static void free_list(struct Node *head) {
    while (head != NULL) {
        struct Node *next = head->next;
        free(head);
        head = next;
    }
}

int main(void) {
    int n;
    printf("Enter the number of elements: ");
    if (scanf("%d", &n) != 1 || n < 0) return 1;

    struct Node *head = NULL;
    printf("Enter the elements:\n");
    for (int i = 0; i < n; ++i) {
        int value;
        scanf("%d", &value);
        head = insert(head, value);
    }

    display(head);
    free_list(head);
}

```

```
    return 0;
}
```

Linked-List/linked_list_length.c

```
#include <stdio.h>
#include <stdlib.h>

struct node {
    int data;
    struct node *next;
};

int main(void) {
    struct node *head = malloc(sizeof(struct node));
    if (head == NULL) return 1;

    head->data = 2;
    head->next = malloc(sizeof(struct node));
    if (head->next == NULL) {
        free(head);
        return 1;
    }

    head->next->data = 3;
    head->next->next = malloc(sizeof(struct node));
    if (head->next->next == NULL) {
        free(head->next);
        free(head);
        return 1;
    }

    head->next->next->data = 4;
    head->next->next->next = NULL;

    int length = 0;
    struct node *temp = head;
    while (temp != NULL) {
        ++length;
        temp = temp->next;
    }

    printf("Length of the linked list: %d\n", length);

    temp = head;
    while (temp != NULL) {
        struct node *next = temp->next;
        free(temp);
        temp = next;
    }
}
```

```

    }
    return 0;
}

```

Strings/longest_word.c

```

#include <stdio.h>
#include <string.h>

int main(void) {
    char str[501], longest[501] = "";
    int words = 0, current = 0, max = 0, start = 0;

    printf("Enter string: ");
    if (!fgets(str, sizeof(str), stdin)) return 1;

    size_t len = strlen(str);
    for (size_t i = 0; i <= len; ++i) {
        if (str[i] != ' ' && str[i] != '\n' && str[i] != '\0') {
            ++current;
        } else if (current > 0) {
            ++words;
            if (current > max) {
                max = current;
                memcpy(longest, &str[start], (size_t)current);
                longest[current] = '\0';
            }
            current = 0;
            start = (int)i + 1;
        }
    }

    printf("Number of words: %d\n", words);
    printf("Length of the longest word: %d\n", max);
    printf("Longest word: %s\n", longest);
    return 0;
}

```

Strings/remove_duplicates.c

```

#include <stdio.h>
#include <string.h>

int main(void) {
    int n;
    scanf("%d", &n);
    char entries[100][101];

    for (int i = 0; i < n; ++i)

```

```

        scanf("%100s", entries[i]);

int count = 0;
for (int i = 0; i < n; ++i) {
    int duplicate = 0;
    for (int j = 0; j < i; ++j) {
        if (strcmp(entries[i], entries[j]) == 0) {
            duplicate = 1;
            break;
        }
    }
    if (!duplicate) ++count;
}

printf("%d\n", count);
for (int i = 0; i < n; ++i) {
    int duplicate = 0;
    for (int j = 0; j < i; ++j)
        if (strcmp(entries[i], entries[j]) == 0)
            duplicate = 1;
    if (!duplicate)
        printf("%s\n", entries[i]);
}
}

```

Strings/reverse_and_vowels.c

```

#include <stdio.h>
#include <string.h>

int main(void) {
    char word[101];

    printf("Please enter string: ");
    if (scanf("%100s", word) != 1) return 1;

    size_t length = strlen(word);
    for (size_t i = 0; i < length / 2; ++i) {
        char temp = word[i];
        word[i] = word[length - i - 1];
        word[length - i - 1] = temp;
    }

    printf("Transformed string: ");
    for (size_t i = 0; i < length; ++i) {
        char c = word[i];
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            printf("%d", c);
        else

```

```

        putchar(c);
    }
    printf("\nLength: %zu\n", length);
    return 0;
}

```

Structures/employees_departments.c

```

#include <stdio.h>
#include <string.h>

typedef struct {
    int id;
    char name[50];
    int dept_id;
} Employee;

typedef struct {
    int id;
    char name[50];
    int manager_id;
} Department;

Employee employees[100];
Department departments[100];
int emp_count = 0, dept_count = 0;

void add_employee(int id, const char *name, int dept_id) {
    employees[emp_count].id = id;
    strcpy(employees[emp_count].name, name);
    employees[emp_count].dept_id = dept_id;
    ++emp_count;
}

void add_department(int id, const char *name) {
    departments[dept_count].id = id;
    strcpy(departments[dept_count].name, name);
    departments[dept_count].manager_id = -1;
    ++dept_count;
}

void assign_manager(int dept_id, int manager_id) {
    for (int i = 0; i < dept_count; ++i) {
        if (departments[i].id == dept_id) {
            departments[i].manager_id = manager_id;
            return;
        }
    }
}

```

```
int main(void) {
    add_employee(101, "john", 201);
    add_employee(102, "alice", 202);
    add_employee(103, "mark", 202);
    add_department(201, "sales");
    add_department(202, "marketing");
    assign_manager(201, 101);
    assign_manager(202, 102);

    for (int i = 0; i < emp_count; ++i)
        printf("%d %s %d\n", employees[i].id, employees[i].name, employees[i].d);
    return 0;
}
```